Operating System Lab Notes

Ideal for OS Lab Viva

Prepared by: Mr. Sumit Garg

1. pwd – Print Working Directory

Shows the **full absolute path** of the current directory where the user is working.

Why it is used

- To know "where you are" in the Linux file system.
- Useful before navigating, copying, or running scripts.

Syntax

pwd

Example

/home/sumit/Documents

Common mistakes

- Students think pwd shows all directories. It only shows *current* directory.

2. ls – List Directory Contents

Purpose

Displays files and directories present in the current folder.

Syntax

ls

ls [options]

Most useful options

Option	Meaning
ls -l	Long listing (permissions, size, owner, date)
ls -a	Shows **hidden files** (starting with .)
ls -lh	Human readable sizes (KB, MB)
ls -R	Recursive listing (subdirectories too)

Example

ls -l

Output:

-rw-r--r-- 1 sumit users 120 Jan 10 file.txt

drwxr-xr-x 2 sumit users 4096 Jan 10 scripts/

Common mistakes

- ls -al is often written as ls -a -l. Combine like ls -al.

3. cd – Change Directory

Purpose

Used to move between directories.

Syntax

cd directory_name

cd ..

cd ~

cd /

Examples

cd /home/sumit/Documents

cd .. # go to parent directory

cd # takes you to home directory

cd / # go to root

Common mistake

- Using backslashes (\) like Windows. Linux uses **forward slash (/)**.

4. mkdir – Make Directory

Purpose

Creates new folders.

Syntax

mkdir foldername

mkdir folder1 folder2

mkdir -p a/b/c # create nested directories

Example

mkdir lab1

Common mistake

- Forgetting -p leads to errors when parent directories do not exist.

5. rmdir – Remove Directory

Purpose

Deletes **empty directories only**.

Syntax

rmdir dirname

Example

rmdir lab1

Important

- To delete non-empty directories: rm -r dir

6. touch – Create Empty File / Update Timestamp

Purpose

- Creates an empty file
- Updates access/modified date of existing file

Syntax

touch filename

touch file1 file2

Example

touch notes.txt

7. cat – Display, Create, Merge Files

Purpose

Used to:

- Display file contents
- Create file
- Append to file
- Merge multiple files

Syntax

cat file

cat > file # create/overwrite

cat >> file # append

cat file1 file2 > newfile

Example

cat > test.txt

Hello world

Ctrl + D

Danger

- cat > file **overwrites** file completely.

8. cp – Copy File or Directory

Syntax

cp source destination

cp file1.txt backup/

cp -r dir1 dir2 # copy directory

Example

cp notes.txt notes_backup.txt

9. mv – Move or Rename File

Syntax

mv oldname newname

mv file.txt /home/sumit/

Example

mv a.txt b.txt # rename

10. rm – Remove/Delete File

Syntax

rm file

rm -r directory

rm -f file # force delete

Warning

Irreversible—no recycle bin.

11. echo – Print Output

Purpose

- Display strings
- Print variables
- Write into file

Syntax

echo text

echo $variable

echo "hello" > file.txt

12. wc – Word Count

Purpose

Counts:

- Lines
- Words
- Characters

Syntax

wc file

wc -l file # count lines

wc -w file # count words

wc -c file # count characters

13. grep – Search for Pattern in File

Purpose

Searches text using patterns.

Syntax

grep "word" file

grep -i "word" file # ignore case

grep -n "word" file # line numbers

14. cut – Select Columns

Purpose

Extract specific columns from a file.

Syntax

cut -d "delimiter" -f field file

Example

cut -d ":" -f1 /etc/passwd

15. sort – Sort Lines Alphabetically

Syntax

sort file

sort -r file # reverse sort

sort -n file # numeric sort

sort file1 > sorted.txt

16. head and tail

head → shows first lines

head -n 5 file.txt

tail → shows last lines

tail -n 5 file.txt

tail -f logfile.log # live monitoring

17. df – Disk Free Space

df -h # human readable

18. du – Directory Size

du -h

du -sh folder

19. who, whoami, id

who # logged-in users

whoami # current username

id # UID, GID

20. man – Manual Pages

man ls

man grep

Shows official documentation.

21. history – View Previously Used Commands

history

!5 # repeat command no. 5

22. chmod – Change File Permissions

chmod 755 file.sh

chmod +x script.sh

23. cal – Display Calendar

cal

cal 8 2025

2. Shell Script Introduction

Definition:
A shell script is a text file containing UNIX/Linux commands executed in sequence by the shell interpreter.

Basic script structure:

```
#!/bin/bash

# This is a comment

echo "Hello World"
```

Steps to run a script:

1. Create file: vi script.sh
2. Make it executable: chmod +x script.sh
3. Execute: ./script.sh

Uses of shell scripts:
Automation, backups, monitoring, user-input processing, string processing, mathematical calculations, file management.

Common shell types:
sh, bash, ksh, csh.

3. Variables in Shell

Variables store values such as numbers or text.

Syntax:

```
varname=value
```

No spaces are allowed on either side of =.

Access variable:

```
echo $varname
```

Taking input:

```
echo "Enter name:"

read name
```

or

```
read -p “Enter name: “ name

echo "Your name is $name"
```

System variables:

```
echo $USER
```

echo $HOME

echo $PWD

echo $SHELL

Unset a variable:

unset varname

Make variable read-only:

readonly varname

4. Command Line Arguments (CLA)

Definition:
Arguments passed when executing the script.

Syntax:

./script.sh arg1 arg2

5. Operators in Shell

Arithmetic operators :

$((5 + 3))

$((10 – 4))

$((4 * 3))

$((8 / 2))

$((9 % 2))

Relational operators (for numeric comparison):

-eq Equal to

-ne Not equal

-gt Greater than

-lt Less than

-ge Greater or equal

-le Less or equal

Example:

```
if [ $a -gt $b ]
then
  echo "a is greater"
```

fi

String comparison operators:

= Equal

!= Not equal

-z String is empty

-n String is not empty

Logical operators:

-a Logical AND

-o Logical OR

! Logical NOT

6. Conditional Statements

if statement:

```
if [ condition ]
then
  commands
fi
```

if-else:

```
if [ condition ]
then
  commands
else
  other commands
fi
```

else-if ladder:

```
if [ $marks -ge 90 ]
then
  echo "A grade"
elif [ $marks -ge 75 ]
then
  echo "B grade"
else
```

```
  echo "C grade"
fi
```

Case statement:

```
case $choice in
1) echo "Option 1" ;;
2) echo "Option 2" ;;
*) echo "Invalid" ;;
esac
```

7. Looping Statements

for loop:

```
for i in 1 2 3 4
do
  echo $i
done
```

for loop using range :

```
for i in {1..10}
do
  echo $i
done
```

while loop:

```
count=1
while [ $count -le 5 ]
do
  echo $count
  count=`expr $count + 1`
done
```

until loop:

```
num=1
until [ $num -gt 5 ]
do
  echo $num
```

```
  num=`expr $num + 1`
done
```

8. Input/Output Redirection

Overwrite contents:

command > file

Append:

command >> file

Take input from file:

command < file

9. Pipes in Shell

Definition:
Pipe transfers output of one command as input to another.

Syntax:

command1 | command2

Common pipeline commands:

grep search text

sort sort lines

uniq remove duplicates

wc count lines, words, characters

cut extract columns

head first lines

tail last lines

Examples:

cat file | sort | uniq

ls -l | wc -l

10. Strings in Shell Script

Declaring a string:

str="Hello World"

Displaying:

```
echo $str
```

String length:

```
echo ${#str}
```

Substring extraction:

```
echo ${str:0:5}     # Hello
echo ${str:6}       # World
```

String comparison:

```
if [ "$a" = "$b" ]
then
  echo "equal"
fi
```

Concatenation:

```
str3="$str1 $str2"
```

Reverse string:

```
echo $str | rev
```

Check empty string:

```
if [ -z "$str" ]
then
  echo "Empty"
fi
```

Replace substring:

```
file="hello.txt"
echo ${file/.txt/.pdf}
```

11. Arrays in Shell Script

Array declaration:

```
arr=(10 20 30 40)
```

Access elements:

```
echo ${arr[0]}
echo ${arr[3]}
```

Print all elements:

```
echo ${arr[@]}
echo ${arr[*]}
```

Array length:

```
echo ${#arr[@]}
```

Replace one element:

```
arr[2]=99
```

Loop through array:

```
for val in ${arr[@]}
do
  echo $val
done
```

Array index loop:

```
for i in ${!arr[@]}
do
  echo "Index: $i Value: ${arr[$i]}"
done
```

Sorting array (manual bubble sort):

```
arr=(9 2 6 1)
for ((i=0;i<4;i++))
do
 for ((j=0;j<4-i-1;j++))
 do
   if [ ${arr[j]} -gt ${arr[$((j+1))]} ]
   then
     temp=${arr[j]}
     arr[$j]=${arr[$((j+1))]}
     arr[$((j+1))]=$temp
   fi
 done
done
echo ${arr[@]}
```

12. Basic Shell Script Patterns

Print message:

```
echo "Welcome to Shell Scripting"
```

Check a number:

```
if [ $n -gt 0 ]
then
  echo "Positive"
else
  echo "Negative"
fi
```

Sum of two numbers:

```
sum=$(( $a + $b ))
echo $sum
```

Loop with commands:

```
for file in *.txt
do
   echo $file
done
```

`